

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

Pertemuan 13

Fitur Lanjutan, Performa, dan Tes Lisan 2

Menguji alur LaporKampus dari form sampai daftar, membuat daftar berfoto lancar digulir, lalu menunjukkannya di tes lisan 2

Muhammad Habib Algifari, S.Kom., M.T.I.

Tahun 2026



Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera

Tiga pola dari refleksi tes lisan 1

Pertanyaan refleksi	Pola yang sering muncul
Bagian kode mana yang paling sulit kalian jelaskan saat tes lisan?	Penjelasan arsitektur: alur dari layar ke ViewModel ke repository, terutama siapa yang boleh mengubah state
Satu masukan dari kelompok lain yang akan kalian kerjakan?	Data contoh: terlalu sedikit atau tidak masuk akal, misalnya judul tes123 dan lokasi kosong
Apa langkah pertama kalian saat aplikasi crash?	Crash paling sering muncul saat mode pesawat. Langkah yang tepat: Logcat, Caused by, baris paket sendiri

Daftar yang patah-patah

Daftar laporan terasa patah-patah saat digulir setelah ada foto. Fitur kamera dan saran AI kelompok juga belum benar-benar tersambung ke alur aplikasi, masih di layar uji masing-masing.

DISKUSI 2 MENIT

Kenapa daftar dua puluh laporan bisa patah-patah di HP yang cukup baru? Dan bagaimana kita membuktikan foto dan saran AI benar-benar ikut sampai laporan muncul di daftar?

ui/daftar/LayarDaftar.kt, versi sekarang

```
@Composable
fun LayarDaftar(vm: DaftarViewModel = koinViewModel()) {
    val daftar by vm.daftar.collectAsState()
    LazyColumn {
        items(daftar.sortedByDescending { it.id }) {
            KartuLaporan(it) // foto asli 4000 x 3000
        }
    }
}
```

Saat pulang nanti, kalian bisa

1 **Menguji lintas lapisan**
Menulis integration test dengan MockEngine dan database di memori, dari form sampai daftar.

2 **Memperbaiki performa**
Mengukur recomposition, lalu memakai key, foto kecil, dan data yang stabil.

3 **Menyambungkan fitur**
Membawa kamera dan saran AI ke alur form, plus pencarian atau tarik untuk menyegarkan.

Capaian ini mendukung **CPMK0501**: menerapkan konsep pemrograman untuk pengembangan perangkat lunak, dalam bentuk aplikasi yang fiturnya tersambung, teruji, dan lancar dipakai.

Bagian 1

Integration test

Merakit beberapa lapisan asli sekaligus, dan hanya memalsukan yang ada di luar aplikasi.

Unit, integration, dan UI test

	Unit test	Integration test	UI test
Yang diuji	Satu kelas, misalnya FormLaporanViewModel	Beberapa lapisan bersama: ViewModel, repository, Ktor, SQL	Layar dan interaksi pengguna
Yang asli	Hanya kelas itu	Semua kode kita dari ViewModel sampai query	Composable dan state-nya
Yang palsu	Semua dependensinya, misalnya SumberLaporanPalsu	Hanya batas luar: server dan berkas database	Data contoh
Kecepatan	Paling cepat	Cepat, tetap tanpa internet	Paling lambat

Server palsu dengan MockEngine

commonMain/data/remote/Jaringan.kt

```
fun buatClient(engine: HttpClientEngine) =  
    HttpClient(engine) {  
        install(ContentNegotiation) {  
            json(Json { ignoreUnknownKeys = true })  
        }  
        expectSuccess = true  
    }
```

Dependency di build.gradle.kts

```
commonTest.dependencies {  
    implementation(  
        "io.ktor:ktor-client-mock:$ktor")  
    }
```

desktopTest/AlurTambahLaporanTest.kt

```
private const val JAWABAN = ""  
    {"id": 31, "judul": "AC mati",  
     "lokasi": "GKU 1 R201", "status": "Baru"}""  
  
val serverPalsu = MockEngine {  
    respond(  
        content = JAWABAN,  
        status = HttpStatusCode.Created,  
        headers = headersOf(  
            HttpHeaders.ContentType, "application/json")  
        )  
    }
```

Satu-satunya perubahan di kode aplikasi: `buatClient` menerima engine dari luar. Aplikasi memberi engine asli, test memberi `MockEngine`.

Database di memori

desktopTest/AlurTambahLaporanTest.kt

```
fun buatRepoUji(): LaporanRepository {  
    val driver = JdbcSqliteDriver(  
        JdbcSqliteDriver.IN_MEMORY)  
    LaporanKampusDb.Schema.create(driver)  
    return LaporanRepository(  
        api = LaporanApi(buatClient(serverPalsu)),  
        db = LaporanKampusDb(driver)  
    )  
}
```

Folder test	Isinya
commonTest	Test yang jalan di semua platform. MockEngine boleh di sini.
desktopTest	Test yang butuh JVM, seperti JdbcSqliteDriver.

`./gradlew :composeApp:desktopTest` menjalankan keduanya, sama dengan perintah di CI minggu lalu.

Test alur: form sampai daftar

desktopTest/AlurTambahLaporanTest.kt, di dalam class

```
@BeforeTest fun mulai() =  
    Dispatchers.setMain(UnconfinedTestDispatcher())  
@AfterTest fun selesai() = Dispatchers.resetMain()  
  
@Test  
fun laporanBaruMunculDiDaftar() = runTest {  
    val repo = buatRepoUji()  
    val form = FormLaporanViewModel(  
        repo, KameraPalsu(), AsistenPalsu())  
    val daftar = DaftarViewModel(repo)  
    form.ubahJudul("AC mati")  
    form.ubahLokasi("GKU 1 R201")  
    form.kirim()  
    val hasil = daftar.daftar.first { it.isNotEmpty() }  
    assertEquals(31, hasil.single().id)  
    assertEquals("AC mati", hasil.single().judul)  
}
```

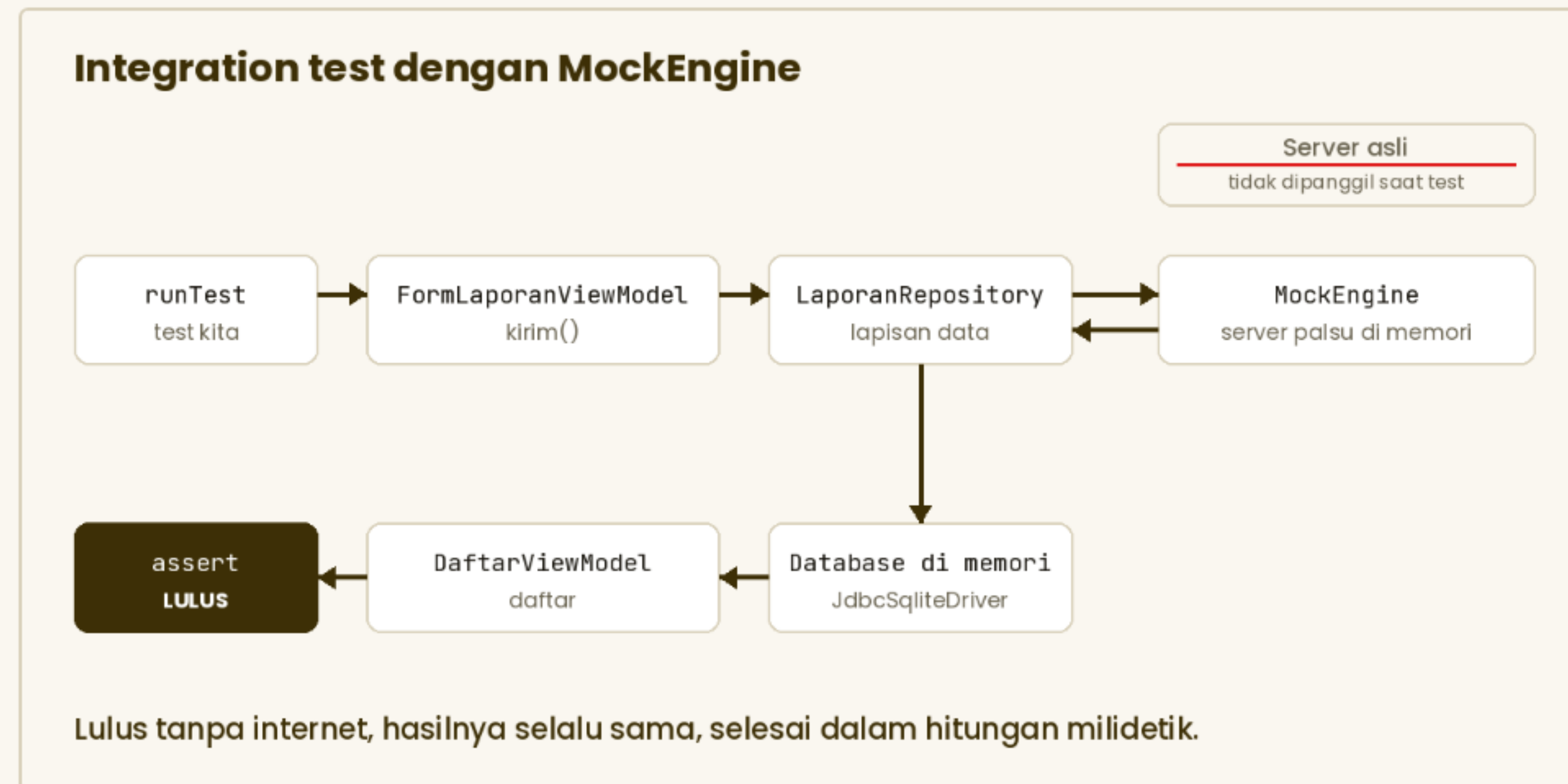
Baris	Artinya
setMain	viewModelScope butuh dispatcher Main, di test kita ganti
first { }	Tunggu sampai query database memancarkan daftar berisi
id 31	Hanya bisa datang dari jawaban server palsu

BANGUN BARENG

Test ini kita jalankan dengan Wi-Fi laptop dimatikan

Lulus atau gagal, dan kira-kira berapa lama? Lalu, apa yang terjadi kalau MockEngine kita ubah agar menjawab 500?

Cek tebakanmu: alur di dalam test



Animasi berputar saat presentasi. Di PDF yang tampil keadaan akhirnya.

COBA LANGSUNG



Scan atau klik untuk membaca cara memakai MockEngine, termasuk memeriksa request yang dikirim.

[Testing in Ktor Client, ktor.io](https://ktor.io/docs/testing-in-ktor-client.html)

Sumber: Dokumentasi Ktor, JetBrains.

Bagian 2

Performa daftar

Ukur dulu, baru perbaiki, lalu ukur lagi.

Tanda jank dan alat ukurnya

Apa itu jank

Layar 60 Hz menggambar satu frame sekitar setiap 16 milidetik. Kalau kerja di satu frame lebih lama, frame itu terlewat dan gulir terasa patah-patah.

Gejalanya: gulir tersendat, kartu muncul terlambat, atau tombol terasa lambat merespons.

Alat	Yang dilihat
Layout Inspector, Show Recomposition Counts	Hitungan composition dan skip setiap composable
Opsi developer, Profile HWUI rendering	Batang per frame, yang melewati garis berarti terlambat
Build release	Kelancaran sebenarnya, karena build debug memang lebih lambat

BANGUN BARENG

Daftar berisi lima laporan tanpa key, lalu Kursi patah masuk paling atas

Berapa kartu yang digambar ulang? Dan berapa kalau setiap item diberi key berupa id laporan?

Cek tebakanmu: hitungan recomposition

Hitungan recomposition di LazyColumn			Laporan baru masuk di atas		
Tanpa key			Dengan key		
items(daftar)			items(daftar, key = { it.id })		
	hitungan			hitungan	
Kursi patah	Baru	2	Kursi patah	Baru	1
AC mati	Baru	2	AC mati	Baru	1
Proyektor buram	Diproses	2	Proyektor buram	Diproses	1
Kran bocor	Baru	2	Kran bocor	Baru	1
Wifi lambat	Selesai	2	Wifi lambat	Selesai	1
Lampu padam	Baru	1	Lampu padam	Baru	1
6 kartu digambar			1 kartu digambar		
5 kartu lama ikut digambar ulang			Kartu lama hanya bergeser		

Animasi berputar saat presentasi. Di PDF yang tampil keadaan akhirnya.

COBA LANGSUNG



Scan atau klik untuk membaca praktik performa Compose, termasuk key, remember, dan derivedStateOf.

[Follow best practices, Android Developers](#)

Sumber: Dokumentasi Jetpack Compose, Android Developers. Prinsipnya sama untuk Compose Multiplatform.

key di LazyColumn

ui/daftar/LayarDaftar.kt

```
LazyColumn {  
    items(  
        items = daftar,  
        key = { it.id }  
    ) { laporan →  
        KartuLaporan(laporan)  
    }  
}
```

Syarat key	Alasannya
Unik	Dua item dengan key sama membuat aplikasi crash
Tetap	id dari server atau database, bukan indeks atau judul
Tipe sederhana	Int, Long, atau String, supaya bisa disimpan Android

Bonus: posisi gulir tetap pada laporan yang sama walau ada data baru di atasnya.

Lima perbaikan yang paling sering dibutuhkan

Perbaikan	Kenapa membantu	Di LaporKampus
key di LazyColumn	Kartu lama tidak digambar ulang saat bergeser	<code>key = { it.id }</code>
Foto diperkecil	Foto 12 MP untuk kartu kecil boros memori dan waktu	Simpan versi kecil saat foto diambil
Kerja berat keluar composable	Badan composable bisa dijalankan berkali-kali	Urutkan dan saring di ViewModel
Data class stabil	Compose bisa melewati kartu yang isinya sama	Semua field <code>val</code>
remember dan derivedStateOf	Hitung sekali, atau hanya saat hasilnya berubah	Tombol Ke atas

Data stabil, remember, dan derivedStateOf

commonMain/data/model/Laporan.kt

```
@Immutable
data class Laporan(
    val id: Int,
    val judul: String,
    val lokasi: String,
    val fotoUrl: String?,
    val status: StatusLaporan = StatusLaporan.Baru
)
```

ui/daftar/LayarDaftar.kt

```
val posisi = rememberLazyListState()
val tampilkanKeAtas by remember {
    derivedStateOf { posisi.firstVisibleItemIndex > 3 }
}
```

Database memancarkan objek **Laporan** baru setiap kali berubah. Kalau tipenya stabil, Compose membandingkan dengan **equals**. Kalau tidak, dengan **===**, jadi semua kartu dianggap berubah.

derivedStateOf membuat tombol hanya digambar ulang saat jawabannya berganti dari false ke true, bukan setiap piksel gulir.

Bagian 3

Fitur lanjutan di alur aplikasi

Kamera dan saran AI masuk ke form, lalu daftar bisa dicari dan disaring.

Kamera dan saran AI di form yang sama

ui/form/FormLaporanViewModel.kt

```
class FormLaporanViewModel(  
    private val repo: LaporanRepository,  
    private val kamera: Kamera,          // minggu 8  
    private val asisten: AsistenLaporan  // minggu 9  
) : ViewModel() {  
    fun ambilFoto() = viewModelScope.launch {  
        val foto = kamera.ambilFoto() ?: return@launch  
        _uiState.update { it.copy(foto = foto) }  
    }  
    fun mintaSaran() = viewModelScope.launch {  
        _uiState.update { it.copy(memintaSaran = true) }  
        val hasil = asisten.sarankan(_uiState.value.deskripsi)  
        _uiState.update { it.copy(memintaSaran = false,  
            saran = hasil.getOrNull(),  
            saranGagal = hasil.isFailure) }  
    }  
}
```

1

Foto

Tombol kamera di form, batal atau izin ditolak tetap aman.

2

Saran AI

Mengisi kategori dan urgensi yang masih bisa diubah.

3

Kirim

Laporan lengkap masuk repository dan muncul di daftar.

Pencarian dan filter di ViewModel

ui/daftar/DaftarViewModel.kt

```
class DaftarViewModel(repo: LaporanRepository) : ViewModel() {  
    private val _kueri = MutableStateFlow("")  
    val kueri = _kueri.asStateFlow()  
    private val _status = MutableStateFlow(FilterStatus.Semua)  
    val status = _status.asStateFlow()  
  
    val daftar = combine(  
        repo.pantauSemua(), _kueri, _status  
    ) { semua, q, s →  
        semua.filter { it.cocok(q) && s.cocok(it) }  
            .sortedByDescending { it.id }  
    }.stateIn(viewModelScope,  
        SharingStarted.WhileSubscribed(5000), emptyList())  
  
    fun ubahKueri(teks: String) { _kueri.value = teks }  
    fun pilihStatus(s: FilterStatus) { _status.value = s }  
}
```

Aturan pencarian

```
fun Laporan.cocok(q: String) =  
    judul.contains(q, ignoreCase = true) ||  
    lokasi.contains(q, ignoreCase = true)
```

combine menghitung ulang setiap kali data, kueri, atau status berubah. Pengurutan juga pindah ke sini.

Kolom cari dan chip status

ui/daftar/LayarDaftar.kt

```
val kueri by vm.kueri.collectAsState()
val status by vm.status.collectAsState()
Column {
    OutlinedTextField(
        value = kueri, onChange = vm::ubahKueri,
        label = { Text("Cari judul atau lokasi") },
        singleLine = true)
    Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
        FilterStatus.entries.forEach { s →
            FilterChip(selected = s == status,
                onClick = { vm.pilihStatus(s) },
                label = { Text(s.name) })
        }
    }
    DaftarLaporan(daftar)
}
```

Pilihan lain: tarik untuk menyegarkan

```
PullToRefreshBox(
    isRefreshing = menyegarkan,
    onRefresh = vm::segarkan
) {
    DaftarLaporan(daftar)
}
```

segarkan meminta data terbaru dari server ke database, sesuai pola offline-first minggu 7.

Bagian 4

Tes lisan 2

Empat menit per kelompok untuk menunjukkan fitur platform dan fitur AI yang sudah menyatu dengan alur aplikasi.

Empat menit per kelompok

1 Dua menit alur utuh
Dari form: ambil foto, minta saran AI, koreksi, kirim, lalu lihat di daftar.

2 Satu menit bukti
Integration test yang lulus dan CI hijau di tab Actions.

3 Satu menit tanya jawab
Saya bertanya ke satu anggota yang dipilih acak.

Yang dinilai

Fitur platform tema terintegrasi dan teruji, serta fitur AI dengan keluaran terstruktur yang bisa dikoreksi.

Urutan diundi sekarang. Aplikasi sudah terpasang sebelum giliran, kelompok berikutnya bersiap di depan.

Rubrik tes lisan 2: fitur lanjutan

Kriteria	Sangat baik 85 sampai 100	Baik 70 sampai 84	Cukup 55 sampai 69	Kurang di bawah 55
Fitur lanjutan terintegrasi CPMK0501, 5% Kelompok	Fitur platform dan fitur AI tema terpasang di alur aplikasi dan lulus integration test	Keduanya terpasang, belum diuji	Hanya satu yang terpasang, atau berjalan terpisah dari alur aplikasi	Belum ada fitur lanjutan yang terpasang

Tes Lisan (Tugas Kelompok), bobot 5% nilai akhir: CPMK0501 5%. Nilai kriteria dikalikan bobotnya. Plagiat bernilai 0.

Selama sesi berlangsung

Yang maju

Satu orang memegang HP, satu orang menjelaskan, semua siap ditanya.

Yang bersiap

Kelompok berikutnya menyalakan aplikasi dan membuka tab Actions di laptop.

Penonton

Catat satu hal baik dan satu saran untuk setiap kelompok di formulir LMS.

Saran dari penonton dikirim ke kelompok yang dinilai setelah kelas, tanpa nama penulis.

Bagian 5

Coba sendiri, lalu bawa ke proyekmu

Tiga latihan di aplikasi kelompok, checkpoint minggu 13, rubrik tes lisan 3, dan tiket keluar.

Tiga latihan di aplikasimu

Latihan 1 Integration test

Tulis `AlurTambahLaporanTest.kt` versi tema kalian di `desktopTest` dengan `MockEngine` dan database di memori. Selesai kalau lulus dengan Wi-Fi mati.

Latihan 2 Ukur dan perbaiki

Catat hitungan recomposition saat data baru masuk, pasang key dan foto kecil, lalu ukur lagi. Tulis kedua angkanya di deskripsi pull request.

Latihan 3 Fitur lanjutan

Pencarian dan filter, atau tarik untuk menyegarkan. Tambahkan satu unit test untuk logika filternya.

Satu latihan per anggota, di branch sendiri lalu pull request. Pola `@BeforeTest` dan `@AfterTest` sama dengan `KoinModuleTest.kt` di hands-on P11. Belum punya daftar panjang? Pakai `Handson3OfflineNotes.kt` di hands-on P7.

Kalau error

Gejala atau pesan	Penyebab	Yang dicoba
Unresolved reference: MockEngine	ktor-client-mock belum ada di commonTest	Tambahkan dengan versi Ktor yang sama
Unresolved reference: JdbcSqliteDriver	Test ada di commonTest, driver hanya untuk JVM	Pindahkan ke src/desktopTest
Module with the Main dispatcher is missing	viewModelScope butuh dispatcher Main	<code>Dispatchers.setMain</code> di <code>@BeforeTest</code>
no such table: Laporan	Skema belum dibuat di database memori	<code>LaporKampusDb.Schema.create(driver)</code>
<code>NoTransformationFoundException</code>	Jawaban MockEngine tanpa Content-Type JSON	<code>headersOf(HttpHeaders.ContentType, ...)</code>
Key was already used	Dua item dengan id sama, sering dari draf	Pakai id lokal yang tidak bentrok

Sprint minggu ini untuk kelompokmu

- 1** **Satu integration test**
Alur utama tema dari form sampai daftar, lulus di CI.

- 2** **Daftar yang lancar**
key, foto atau gambar kecil, dan kerja berat di ViewModel.

- 3** **Satu fitur lanjutan**
Pencarian dan filter, atau tarik untuk menyegarkan, dengan unit test.

- 4** **Saran tes lisan**
Setiap saran dari sesi tadi menjadi issue di GitHub.

CONTOH DARI LAPORKAMPUS

AlurTambahLaporanTest lulus di CI. LayarDaftar memakai `key = { it.id }` dan foto 300 piksel. Chip Baru, Diproses, Selesai di atas daftar.

Checkpoint proyek minggu 13

1 **Fitur lanjutan**
Pencarian dan filter, atau tarik untuk menyegarkan, di layar daftar.

2 **Integration test**
Minimal satu test alur dengan MockEngine dan database di memori, lulus di CI.

3 **Performa daftar**
key di LazyColumn dan perbaikan yang terukur dengan hitungan recomposition.

Dikumpulkan

Push ke repository kelompok. README memuat cara menjalankan test dan angka recomposition sebelum dan sesudah.

Dicek di awal pertemuan berikutnya: tuntas, sebagian, atau belum.

Rubrik tes lisan 3: stabil dan terdokumentasi

Kriteria	Sangat baik 85 sampai 100	Baik 70 sampai 84	Cukup 55 sampai 69	Kurang di bawah 55
Stabil dan terdokumentasi CPMK0501, 5% Kelompok	Alur utama tanpa crash, test lulus di CI, README memuat deskripsi, cara menjalankan, dan tangkapan layar	Stabil, dokumentasi kurang satu bagian	Ada crash di alur utama	Tidak bisa di-build

Tes Lisan (Tugas Kelompok), bobot 5% nilai akhir: CPMK0501 5%. Nilai kriteria dikalikan bobotnya. Plagiat bernilai 0.

TUTUP

Tiket keluar



1

Jenis test

Apa beda unit test dan integration test?

2

LazyColumn

Kenapa key di LazyColumn membantu performa?

3

Perbaikan

Apa satu perbaikan performa yang kalian lakukan minggu ini?

Jawab singkat di kertas atau formulir LMS sebelum keluar kelas.

TUTUP

Minggu depan: UI/UX, aksesibilitas, dan tes lisan 3



MASALAH MINGGU 14

Aplikasi sudah jalan, tetapi teksnya kecil, kontrasnya rendah di bawah terik matahari, dan pembaca layar tidak tahu arti tombol ikon. Bagaimana membuatnya nyaman dipakai semua orang?

Persiapan

Tes lisan 3: alur utama tanpa crash, CI hijau, dan README lengkap. Coba nyalakan TalkBack atau VoiceOver sekali di HP kalian.

Terima kasih

Sampai jumpa di pertemuan 14

Ada pertanyaan? Sampaikan sekarang, lewat LMS, atau email ke muhammad.algifari@if.itera.ac.id